ABCI Architecture: Deep Dive into Cosmos SDK's Application Blockchain Interface

목차

- 개요
- ABCI란 무엇인가?
- ABCI 스펙 완전 이해
- ABCI++ 개선사항
- BaseApp 아키텍처
- Transaction Execution Flow
- 실전 코드 분석
- 다이어그램
- Best Practices
- 참고 자료

개요

ABCI (Application Blockchain Interface)는 Cosmos SDK 아키텍처의 핵심입니다. 이는 블록체인의 합의 레이어(CometBFT)와 애플리케이션 레이어(Cosmos SDK)를 분리하는 인터페이스입니다.

학습 목표:

- ABCI의 역할과 중요성 이해
- 각 ABCI 메소드의 정확한 동작 방식 파악
- ABCI++의 개선사항과 그 의미 이해
- BaseApp이 ABCI를 어떻게 구현하는지 분석

예상 학습 시간: 3-4시간

ABCI란 무엇인가?

기본 개념

ABCI는 **합의 엔진(Consensus Engine)**과 **상태 머신(State Machine)**을 연결하는 프로토콜입니다.

```
┌──────────────────────────────────────┐
│        Application Layer             │
│    (Cosmos SDK / Your App)           │
│                                      │
│  State Machine: 트랜잭션 실행,        │
│                 상태 변경, 쿼리 처리   │
└──────────────────────────────────────┘
                    ▲
                    │ ABCI Interface
                    │ (Protobuf over gRPC/Socket)
                    ▼
┌──────────────────────────────────────┐
│       Consensus Layer                │
│         (CometBFT)                   │
│                                      │
│  Consensus: 블록 생성, 검증,          │
│             네트워크 통신, P2P        │
└──────────────────────────────────────┘
```

왜 ABCI가 필요한가?

1. 관심사의 분리 (Separation of Concerns)

- **합의 레이어**: "어떤 순서로 트랜잭션을 실행할 것인가?"
- **애플리케이션 레이어**: "트랜잭션이 무엇을 하는가?"

2. 재사용성

- 하나의 합의 엔진(CometBFT)을 다양한 애플리케이션에서 사용
- 다른 합의 엔진으로 교체 가능 (이론적으로)

3. 언어 중립성

- ABCI는 언어 독립적인 프로토콜
- 어떤 언어로든 블록체인 애플리케이션 구현 가능
- Cosmos SDK는 Go 구현체

ABCI 통신 방식

ABCI는 세 가지 연결 방식을 지원합니다:

1. **Socket (TCP)**: 프로덕션 환경
2. **gRPC**: 고성능이 필요한 경우
3. **In-process**: 개발 및 테스트

ABCI 스펙 완전 이해

ABCI는 여러 메소드로 구성됩니다. 크게 네 가지 카테고리로 분류됩니다:

1. Consensus Connection (합의 관련)

블록 생성 및 실행과 관련된 메소드들입니다.

InitChain

호출 시점: 제네시스(최초 블록) 생성 시 한 번만 호출

목적: 체인 초기화

```
message RequestInitChain {
  google.protobuf.Timestamp time = 1;
  string chain_id = 2;
  ConsensusParams consensus_params = 3;
  repeated ValidatorUpdate validators = 4;
  bytes app_state_bytes = 5;
  int64 initial_height = 6;
}

message ResponseInitChain {
  ConsensusParams consensus_params = 1;
  repeated ValidatorUpdate validators = 2;
  bytes app_hash = 3;
}
```

주요 역할:

- Genesis state 로드
- 초기 validator 세트 설정
- 컨센서스 파라미터 초기화

BaseApp에서의 구현:

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) InitChain(req *abci.RequestInitChain) (*abci.ResponseInitChain, error) {
    // Genesis state 언마샬링
    initState := app.initFromGenesisState(req.AppStateBytes)

    // 각 모듈의 InitGenesis 호출
    if app.initChainer != nil {
        res, err := app.initChainer(app.deliverState.ctx, req)
        if err != nil {
            return nil, err
        }
        return res, nil
    }

    return &abci.ResponseInitChain{}, nil
}
```

FinalizeBlock (ABCI++ / Cosmos SDK v0.50+)

호출 시점: 블록이 확정(decided)되었을 때

목적: 블록 실행 및 상태 변경 (BeginBlock + DeliverTx + EndBlock 통합)

> **중요**: Cosmos SDK v0.50부터 BeginBlock, DeliverTx, EndBlock은 **더 이상 개별 ABCI 메소드가 아닙니다**. 이들은 FinalizeBlock 내부에서 자동으로 순차 실행됩니다.

```
message RequestFinalizeBlock {
  repeated bytes txs = 1;
  CommitInfo decided_last_commit = 2;
  repeated Misbehavior misbehavior = 3;
  bytes hash = 4;
  int64 height = 5;
  google.protobuf.Timestamp time = 6;
  bytes next_validators_hash = 7;
  bytes proposer_address = 8;
}

message ResponseFinalizeBlock {
  repeated ExecTxResult tx_results = 1;
  repeated Event events = 2;
  repeated ValidatorUpdate validator_updates = 3;
  ConsensusParams consensus_param_updates = 4;
  bytes app_hash = 5;
}
```

FinalizeBlock 실행 순서:

```
FinalizeBlock
  ↓
1. PreBlock (v0.50+)
  ↓
2. BeginBlock (내부 호출)
  - 블록 헤더 정보 처리
  - Byzantine validator 증거 처리
  - 모듈별 BeginBlocker 실행
  ↓
3. DeliverTx (각 트랜잭션마다)
  - Decode: 트랜잭션 바이트 → Tx 객체
  - Validate: 기본 검증 (signatures, fees)
  - AnteHandler: 전처리 (gas, sequence 확인)
  - MsgServiceRouter: 메시지 라우팅
  - Msg Handler: 비즈니스 로직 실행
  - PostHandler: 후처리 (선택적)
  ↓
4. EndBlock (내부 호출)
  - Validator 업데이트
  - 블록 종료 로직
  ↓
5. Return ResponseFinalizeBlock
```

BaseApp에서의 구현:

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) FinalizeBlock(req *abci.RequestFinalizeBlock) (*abci.ResponseFinalizeBlock, error) {
    // 헤더 검증
    if err := app.validateFinalizeBlockHeight(req); err != nil {
        return nil, err
    }

    // Context 생성
    ctx := app.getContextForFinalizeBlock(req.Height, req.Time, req.Hash)

    // 내부 실행 (PreBlock → BeginBlock → DeliverTxs → EndBlock)
    res, err := app.internalFinalizeBlock(ctx, req)
    if err != nil {
        return nil, err
    }

    return res, nil
}

// 내부 실행 로직
func (app *BaseApp) internalFinalizeBlock(ctx sdk.Context, req *abci.RequestFinalizeBlock) (*abci.ResponseFinalizeBlock, error) {
    // 1. PreBlock (합의 파라미터 수정 등)
    if app.preBlocker != nil {
        ctx, err := app.preBlocker(ctx, req)
        if err != nil {
            return nil, err
        }
    }

    // 2. BeginBlock
    beginBlockEvents := app.beginBlock(ctx, req)

    // 3. DeliverTxs
    txResults := make([]*abci.ExecTxResult, len(req.Txs))
    for i, tx := range req.Txs {
        result := app.deliverTx(ctx, tx)
        txResults[i] = &result
    }

    // 4. EndBlock
    endBlockEvents, validatorUpdates := app.endBlock(ctx, req)

    // 5. AppHash는 Commit에서 계산됨
    return &abci.ResponseFinalizeBlock{
        TxResults:        txResults,
        Events:           append(beginBlockEvents, endBlockEvents...),
        ValidatorUpdates: validatorUpdates,
    }, nil
}
```

BeginBlock / EndBlock (Legacy 개념, 여전히 내부 사용)

v0.50+에서도 **BeginBlock과 EndBlock은 모듈 개발자에게 여전히 노출**됩니다:

```go
// x/staking/module.go
func (am AppModule) BeginBlock(ctx context.Context) error {
    // Slashing 처리, 블록 리워드 등
    return am.keeper.BeginBlocker(ctx)
}

func (am AppModule) EndBlock(ctx context.Context) ([]abci.ValidatorUpdate, error) {
    // Validator 세트 업데이트
    return am.keeper.EndBlocker(ctx)
}
```

사용 예시:

- **Slashing 모듈 BeginBlock**: 오프라인 validator 처벌
- **Distribution 모듈 BeginBlock**: 블록 리워드 분배
- **Mint 모듈 BeginBlock**: 새로운 토큰 발행
- **Staking 모듈 EndBlock**: Validator 세트 변경 반영

PrepareProposal (ABCI++ 신규 메소드)

호출 시점: Proposer가 블록을 제안하기 전

목적: 블록에 포함될 트랜잭션 재정렬, 추가, 제거

```
message RequestPrepareProposal {
  int64 max_tx_bytes = 1;
  repeated bytes txs = 2;
  ExtendedCommitInfo local_last_commit = 3;
  repeated Misbehavior misbehavior = 4;
  int64 height = 5;
  google.protobuf.Timestamp time = 6;
  bytes next_validators_hash = 7;
  bytes proposer_address = 8;
}

message ResponsePrepareProposal {
  repeated bytes txs = 1; // 수정된 트랜잭션 리스트
}
```

활용 사례:

- MEV 방지를 위한 트랜잭션 재정렬
- 고가스 트랜잭션 우선 처리
- 배치 최적화 (예: Osmosis의 batch auction)

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) PrepareProposal(req *abci.RequestPrepareProposal) (*abci.ResponsePrepareProposal, error) {
    if app.prepareProposal != nil {
        return app.prepareProposal(app.finalizeBlockState.ctx, req)
    }

    // 기본 동작: mempool 트랜잭션 그대로 사용
    return &abci.ResponsePrepareProposal{Txs: req.Txs}, nil
}
```

ProcessProposal (ABCI++ 신규 메소드)

호출 시점: Validator가 제안된 블록을 받았을 때

목적: 제안된 블록의 유효성 검증 (결정론적이어야 함)

```
message RequestProcessProposal {
  repeated bytes txs = 1;
  CommitInfo proposed_last_commit = 2;
  repeated Misbehavior misbehavior = 3;
  bytes hash = 4;
  int64 height = 5;
  google.protobuf.Timestamp time = 6;
  bytes next_validators_hash = 7;
  bytes proposer_address = 8;
}

message ResponseProcessProposal {
  ProposalStatus status = 1; // ACCEPT, REJECT
}

enum ProposalStatus {
  UNKNOWN = 0;
  ACCEPT = 1;
  REJECT = 2;
}
```

검증 예시:

- 트랜잭션 순서가 규칙을 따르는지 확인
- 필수 트랜잭션이 포함되었는지 확인
- 블록 크기 제한 준수 확인

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) ProcessProposal(req *abci.RequestProcessProposal) (*abci.ResponseProcessProposal, error) {
    if app.processProposal != nil {
        return app.processProposal(app.finalizeBlockState.ctx, req)
    }

    // 기본 동작: 모든 제안 수락
    return &abci.ResponseProcessProposal{Status: abci.ResponseProcessProposal_ACCEPT}, nil
}
```

Legacy: DeliverTx (ABCI 1.0, Deprecated)

⚠️ **Cosmos SDK v0.50+에서는 사용되지 않음** (FinalizeBlock으로 통합)

이전 버전에서는 블록 내 각 트랜잭션마다 개별 호출되었습니다:

```
// Legacy (ABCI 0.x)
message RequestDeliverTx {
  bytes tx = 1;
}

message ResponseDeliverTx {
  uint32 code = 1;  // 0 = success
  bytes data = 2;
  string log = 3;
  string info = 4;
  int64 gas_wanted = 5;
  int64 gas_used = 6;
  repeated Event events = 7;
  string codespace = 8;
}
```

Legacy: EndBlock (ABCI 1.0, Deprecated at ABCI level)

⚠️ **ABCI 인터페이스에서는 제거되었지만, 모듈 레벨에서는 여전히 사용**

```
message RequestEndBlock {
  int64 height = 1;
}

message ResponseEndBlock {
  repeated ValidatorUpdate validator_updates = 1;
  ConsensusParams consensus_param_updates = 2;
  repeated Event events = 3;
}
```

주요 역할:

- Validator set 업데이트
- Staking 관련 처리 (unbonding, redelegation)
- 컨센서스 파라미터 업데이트

```
// Legacy code - v0.50+에서는 FinalizeBlock 내부에서 자동 호출
func (app *BaseApp) endBlock(ctx sdk.Context, req *abci.RequestFinalizeBlock) ([]abci.Event, []abci.ValidatorUpdate) {
    // 각 모듈의 EndBlocker 실행
    events, validatorUpdates := app.moduleManager.EndBlock(ctx)
    return events, validatorUpdates
}
```

Commit

호출 시점: FinalizeBlock 이후

목적: 상태를 디스크에 영구 저장하고 AppHash 계산

```
message RequestCommit {}

message ResponseCommit {
  bytes data = 2;  // app_hash
  int64 retain_height = 3;
}
```

주요 역할:

- FinalizeBlock에서 변경된 상태를 MultiStore에 커밋
- IAVL 트리의 새로운 루트 해시 계산 → AppHash
- 메모리 정리 및 다음 블록 준비

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) Commit() (*abci.ResponseCommit, error) {
    // FinalizeBlock 상태 커밋
    header := app.finalizeBlockState.Context().BlockHeader()
    retainHeight := app.GetRetainHeight()

    // MultiStore에 상태 영구 저장
    commitID := app.cms.Commit()

    // App hash는 커밋된 state의 루트 해시
    appHash := commitID.Hash

    // 상태 정리
    app.finalizeBlockState = nil

    return &abci.ResponseCommit{
        Data:         appHash,
        RetainHeight: retainHeight,
    }, nil
}
```

Commit 이후 상태:

```
Before Commit:
  - finalizeBlockState (메모리, 미저장)

After Commit:
  - finalizeBlockState = nil
  - MultiStore에 영구 저장 완료
  - AppHash = IAVL 루트 해시
  - 다음 블록을 위한 checkState 준비
```

2. Mempool Connection

트랜잭션 검증 및 멤풀 관리입니다.

CheckTx

호출 시점: 트랜잭션이 멤풀에 들어가기 전

목적: 트랜잭션이 유효한지 빠르게 검증

```proto
message RequestCheckTx {
  bytes tx = 1;
  CheckTxType type = 2;  // New or Recheck
}

message ResponseCheckTx {
  uint32 code = 1;
  bytes data = 2;
  string log = 3;
  string info = 4;
  int64 gas_wanted = 5;
  int64 gas_used = 6;
  repeated Event events = 7;
  string codespace = 8;
  int64 priority = 10;  // 트랜잭션 우선순위
}
```

CheckTx의 두 가지 타입:

1. **CheckTx_New**: 새로운 트랜잭션
2. **CheckTx_Recheck**: 블록 커밋 후 멤풀 재검증

검증 항목:

- Signature 검증
- Account sequence 확인
- 충분한 gas fee
- 기본적인 유효성 검사

중요: CheckTx는 상태를 변경하지 않습니다! (checkState 사용)

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) CheckTx(req *abci.RequestCheckTx) (*abci.ResponseCheckTx, error) {
    // Tx 디코딩
    tx, err := app.txDecoder(req.Tx)
    if err != nil {
        return &abci.ResponseCheckTx{Code: 1}, err
    }

    // CheckState에서 실행 (격리된 상태, deliverState와 독립적)
    gInfo, result, _, priority, err := app.runTx(runTxModeCheck, req.Tx, tx)
    if err != nil {
        return &abci.ResponseCheckTx{
            Code:      1,
            GasWanted: int64(gInfo.GasWanted),
            GasUsed:   int64(gInfo.GasUsed),
            Log:       err.Error(),
        }, nil
    }

    return &abci.ResponseCheckTx{
        Code:      0,
        GasWanted: int64(gInfo.GasWanted),
        GasUsed:   int64(gInfo.GasUsed),
        Priority:  priority,
        Events:    result.Events,
    }, nil
}
```

3. Info/Query Connection

블록체인 상태 조회입니다.

Query

호출 시점: 클라이언트가 상태를 조회할 때

목적: 블록체인 상태 읽기 (읽기 전용)

```
message RequestQuery {
  bytes data = 1;
  string path = 2;
  int64 height = 3;
  bool prove = 4;
}

message ResponseQuery {
  uint32 code = 1;
  string log = 3;
  string info = 4;
  int64 index = 5;
  bytes key = 6;
  bytes value = 7;
  ProofOps proof_ops = 8;
  int64 height = 9;
  string codespace = 10;
}
```

Query 경로 예시:

- `/app/version` : 앱 버전
- `/store/{storeName}/key` : Store에서 직접 key 조회
- `/custom/{moduleName}/{queryPath}` : 모듈 커스텀 쿼리

```go
// baseapp/abci.go (Cosmos SDK v0.50+)
func (app *BaseApp) Query(ctx context.Context, req *abci.RequestQuery) (*abci.ResponseQuery, error) {
    path := splitPath(req.Path)

    if len(path) == 0 {
        return &abci.ResponseQuery{
            Code: 1,
            Log:  "no query path provided",
        }, nil
    }

    switch path[0] {
    case "app":
        return handleQueryApp(app, req), nil
    case "store":
        return handleQueryStore(app, path, req), nil
    case "p2p":
        return handleQueryP2P(app, path), nil
    case "custom":
        return handleQueryCustom(app, path, req), nil
    }

    return &abci.ResponseQuery{
        Code: 1,
        Log:  fmt.Sprintf("unknown query path: %s", path[0]),
    }, nil
}
```

Info

호출 시점: 체인 정보 요청 시

```
message RequestInfo {
  string version = 1;
  uint64 block_version = 2;
  uint64 p2p_version = 3;
}

message ResponseInfo {
  string data = 1;
  string version = 2;
  uint64 app_version = 3;
  int64 last_block_height = 4;
  bytes last_block_app_hash = 5;
}
```

ABCI++ 개선사항

ABCI++는 CometBFT v0.37+에서 도입된 ABCI의 개선 버전입니다.

주요 개선사항

1. PrepareProposal

새로운 기능: 블록 제안자가 블록을 제안하기 전에 트랜잭션 세트를 수정할 수 있습니다.

```
message RequestPrepareProposal {
  repeated bytes txs = 1;
  int64 max_tx_bytes = 2;
  ExtendedCommitInfo local_last_commit = 3;
  repeated Misbehavior misbehavior = 4;
  int64 height = 5;
  google.protobuf.Timestamp time = 6;
  bytes next_validators_hash = 7;
  string proposer_address = 8;
}

message ResponsePrepareProposal {
  repeated bytes txs = 1;
}
```

사용 사례:

- **MEV 방지**: 트랜잭션 순서 최적화
- **우선순위 처리**: 긴급 거버넌스 트랜잭션 우선 처리
- **배치 처리**: 여러 트랜잭션을 하나로 묶기
- **스팸 필터링**: 의심스러운 트랜잭션 제거

```go
func (app *BaseApp) PrepareProposal(req abci.RequestPrepareProposal) abci.ResponsePrepareProposal {
    // 커스텀 로직으로 트랜잭션 필터링/재정렬
    txs := req.Txs

    // 예: gas price가 높은 순으로 정렬
    sort.Slice(txs, func(i, j int) bool {
        return getGasPrice(txs[i]) > getGasPrice(txs[j])
    })

    // 최대 블록 크기 제한
    selectedTxs := selectTxsUpToMaxBytes(txs, req.MaxTxBytes)

    return abci.ResponsePrepareProposal{
        Txs: selectedTxs,
    }
}
```

2. ProcessProposal

새로운 기능: Validator가 제안된 블록을 accept/reject할 수 있습니다.

```proto
message RequestProcessProposal {
  repeated bytes txs = 1;
  bytes proposed_last_commit = 2;
  repeated Misbehavior misbehavior = 3;
  bytes hash = 4;
  int64 height = 5;
  google.protobuf.Timestamp time = 6;
  bytes next_validators_hash = 7;
  string proposer_address = 8;
}

message ResponseProcessProposal {
  ProposalStatus status = 1;  // Accept or Reject
}
```

사용 사례:

- **악의적 블록 거부**: 부적절한 트랜잭션 포함된 블록 거부
- **프로토콜 규칙 강제**: 애플리케이션 레벨 검증
- **스팸 방지**: 과도한 트랜잭션 포함 시 거부

```go
func (app *BaseApp) ProcessProposal(req abci.RequestProcessProposal) abci.ResponseProcessProposal {
    // 제안된 블록 검증
    for _, tx := range req.Txs {
        if !isValidTx(tx) {
            return abci.ResponseProcessProposal{
                Status: abci.ResponseProcessProposal_REJECT,
            }
        }
    }

    return abci.ResponseProcessProposal{
        Status: abci.ResponseProcessProposal_ACCEPT,
    }
}
```

3. ExtendVote & VerifyVoteExtension

새로운 기능: Validator가 vote에 추가 데이터를 첨부할 수 있습니다.

사용 사례:

- **Oracle 데이터**: 가격 피드를 vote에 포함
- **크로스체인 정보**: 다른 체인의 상태 정보
- **합의 외 데이터 공유**: Validator 간 추가 정보 교환

```
message RequestExtendVote {
  bytes hash = 1;
  int64 height = 2;
}

message ResponseExtendVote {
  bytes vote_extension = 1;
}
```

4. FinalizeBlock (통합 메소드)

개선: BeginBlock, DeliverTx, EndBlock을 하나의 메소드로 통합

```
message RequestFinalizeBlock {
  repeated bytes txs = 1;
  // ... (BeginBlock, EndBlock 정보 포함)
}

message ResponseFinalizeBlock {
  repeated ExecTxResult tx_results = 1;
  // ... (BeginBlock, EndBlock 응답 포함)
}
```

장점:

- **성능 향상**: 3번의 왕복 통신을 1번으로 감소
- **일관성**: 블록 처리 로직을 하나의 atomic 작업으로
- **단순화**: 애플리케이션 로직 단순화

ABCI++ 마이그레이션

Legacy (ABCI 0.x):

```
InitChain
  ↓
[For each block]
  BeginBlock
  [For each tx]
    DeliverTx
  EndBlock
  Commit
```

ABCI++ (ABCI 2.0):

```
InitChain
  ↓
[For each block]
  [Proposer] PrepareProposal
  [Validators] ProcessProposal
  [Validators] ExtendVote
  [Validators] VerifyVoteExtension
  FinalizeBlock (BeginBlock + DeliverTx + EndBlock 통합)
  Commit
```

BaseApp 아키텍처

BaseApp은 Cosmos SDK의 ABCI 구현체입니다.

BaseApp 구조

```go
// baseapp/baseapp.go
type BaseApp struct {
    // 기본 정보
    name                string
    db                  dbm.DB
    cms                 sdk.CommitMultiStore

    // ABCI 핸들러
    initChainer         sdk.InitChainer
    beginBlocker        sdk.BeginBlocker
    endBlocker          sdk.EndBlocker
    anteHandler         sdk.AnteHandler
    postHandler         sdk.PostHandler

    // 라우터
    msgServiceRouter    *MsgServiceRouter
    grpcQueryRouter     *GRPCQueryRouter

    // 실행 상태
    checkState          *state  // CheckTx용 상태
    deliverState        *state  // DeliverTx용 상태
    prepareState        *state  // PrepareProposal용 상태
    processState        *state  // ProcessProposal용 상태

    // 기타
    txDecoder           sdk.TxDecoder
    txEncoder           sdk.TxEncoder
    queryRouter         QueryRouter
    ...
}
```

네 가지 상태 (State)

BaseApp은 네 가지 격리된 상태를 유지합니다 (ABCI++ 이후):

1. checkState

- **용도**: CheckTx 실행
- **생명주기**: Commit 시 리셋
- **특징**: 멤풀 검증용, 실제 상태에 영향 없음

2. deliverState

- **용도**: FinalizeBlock (구 DeliverTx) 실행
- **생명주기**: Commit 시 영구 저장
- **특징**: 실제 상태 변경

3. prepareState (ABCI++ 추가)

- **용도**: PrepareProposal 실행
- **생명주기**: 블록 제안 준비 시 생성, 완료 후 폐기
- **특징**: Proposer가 블록에 포함할 트랜잭션 선별 시 사용

4. processState (ABCI++ 추가)

- **용도**: ProcessProposal 실행
- **생명주기**: 블록 제안 검증 시 생성, 완료 후 폐기
- **특징**: Validator가 제안된 블록 검증 시 사용

> **참고**: `queryState`는 별도 필드가 아닌 마지막 커밋된 상태(`cms`)를 직접 참조합니다.

```
┌──────────────┐
│ prepareState │ ← PrepareProposal (블록 제안 준비, Proposer만)
└──────────────┘

┌──────────────┐
│ processState │ ← ProcessProposal (블록 제안 검증)
└──────────────┘

┌──────────────┐
│  checkState  │ ← CheckTx (멤풀 검증)
└──────────────┘

┌──────────────┐
│ deliverState │ ← FinalizeBlock (실제 실행)
└──────────────┘
       │
       │ Commit
       ↓
┌──────────────┐
│     cms      │ ← Query (조회, 커밋된 상태)
│ (committed)  │
└──────────────┘
```

Router 시스템

MsgServiceRouter

메시지를 적절한 모듈의 Msg 핸들러로 라우팅합니다.

```go
type MsgServiceRouter struct {
    routes map[string]MsgServiceHandler
}

func (msr *MsgServiceRouter) RegisterService(sd *grpc.ServiceDesc, ss interface{}) {
    // Protobuf service descriptor로부터 라우트 생성
    for _, method := range sd.Methods {
        fqName := fmt.Sprintf("/%s/%s", sd.ServiceName, method.MethodName)
        msr.routes[fqName] = createHandler(method, ss)
    }
}
```

예시:

```
/cosmos.bank.v1beta1.Msg/Send → bank 모듈의 Send 핸들러
/cosmos.staking.v1beta1.Msg/Delegate → staking 모듈의 Delegate 핸들러
```

GRPCQueryRouter

gRPC 쿼리를 적절한 모듈의 Query 서비스로 라우팅합니다.

```
type GRPCQueryRouter struct {
    routes map[string]GRPCQueryHandler
}
```

예시:

```
/cosmos.bank.v1beta1.Query/Balance → bank 모듈의 Balance 쿼리
/cosmos.staking.v1beta1.Query/Validators → staking 모듈의 Validators 쿼리
```

AnteHandler Chain

AnteHandler는 트랜잭션 실행 전에 전처리를 수행합니다.

```
type AnteHandler func(ctx Context, tx Tx, simulate bool) (newCtx Context, err error)

type AnteDecorator interface {
    AnteHandle(ctx Context, tx Tx, simulate bool, next AnteHandler) (newCtx Context, err error)
}
```

기본 AnteHandler 체인:

```
SetUpContextDecorator
  ↓
ValidateBasicDecorator (기본 검증)
  ↓
TxTimeoutHeightDecorator (타임아웃 확인)
  ↓
ValidateMemoDecorator (memo 검증)
  ↓
ConsumeGasForTxSizeDecorator (tx 크기에 대한 gas)
  ↓
DeductFeeDecorator (fee 차감)
  ↓
SetPubKeyDecorator (public key 설정)
  ↓
ValidateSigCountDecorator (signature 개수 검증)
  ↓
SigGasConsumeDecorator (signature 검증 gas)
  ↓
SigVerificationDecorator (signature 검증)
  ↓
IncrementSequenceDecorator (sequence 증가)
```

커스텀 AnteDecorator 예시:

```go
type CustomAnteDecorator struct{}

func (cad CustomAnteDecorator) AnteHandle(
    ctx sdk.Context,
    tx sdk.Tx,
    simulate bool,
    next sdk.AnteHandler,
) (sdk.Context, error) {
    // 커스텀 검증 로직
    if err := customValidation(tx); err != nil {
        return ctx, err
    }

    // 다음 decorator로 전달
    return next(ctx, tx, simulate)
}
```

Transaction Execution Flow

RunTx: 핵심 트랜잭션 실행 로직

```go
// baseapp/baseapp.go
func (app *BaseApp) runTx(mode runTxMode, txBytes []byte) (gInfo sdk.GasInfo, result *sdk.Result, err error) {
    // 1. Tx 디코딩
    tx, err := app.txDecoder(txBytes)
    if err != nil {
        return sdk.GasInfo{}, nil, err
    }

    // 2. Context 설정
    var ctx sdk.Context
    switch mode {
    case runTxModeCheck:
        ctx = app.checkState.ctx
    case runTxModeDeliver:
        ctx = app.deliverState.ctx
    case runTxModeSimulate:
        ctx = app.deliverState.ctx.WithMultiStore(app.cms.CacheMultiStore())
    }

    // 3. Gas meter 설정
    ctx = ctx.WithGasMeter(sdk.NewGasMeter(gaslimit))

    // 4. AnteHandler 실행
    defer func() {
        if r := recover(); r != nil {
            // Panic 처리 (주로 OutOfGas)
            err = handlePanic(r)
        }
        gInfo = sdk.GasInfo{
            GasWanted: gasWanted,
            GasUsed:   ctx.GasMeter().GasConsumed(),
        }
    }()

    newCtx, err := app.anteHandler(ctx, tx, mode == runTxModeSimulate)
    if err != nil {
        return gInfo, nil, err
    }
    ctx = newCtx

    // 5. 메시지 실행
    msgResult, err := app.runMsgs(ctx, tx.GetMsgs(), mode)
    if err != nil {
        return gInfo, nil, err
    }

    // 6. PostHandler 실행 (옵션)
    if app.postHandler != nil {
        newCtx, err = app.postHandler(ctx, tx, mode == runTxModeSimulate)
        if err != nil {
            return gInfo, nil, err
        }
        ctx = newCtx
    }

    return gInfo, msgResult, nil
}
```

단계별 상세 분석

Phase 1: Decoding

```go
tx, err := app.txDecoder(txBytes)
```

- 바이트 배열을 Tx 인터페이스로 변환
- Protobuf 역직렬화
- Tx 구조: Msgs, Fee, Signatures, Memo 등

Phase 2: Context Setup

```
ctx := app.getContextForTx(mode, txBytes)
ctx = ctx.WithGasMeter(sdk.NewGasMeter(tx.GetGas()))
```

- 실행 모드에 따라 적절한 context 선택
- Gas meter 초기화
- Context에 블록 정보 포함 (Height, Time, ChainID 등)

Phase 3: AnteHandler Execution

```
newCtx, err := app.anteHandler(ctx, tx, simulate)
```

주요 작업:

- Signature 검증
- Account sequence 확인 및 증가
- Fee 차감
- Tx size에 대한 gas 소비
- Memo 검증

Gas 소비 예시:

```
// Signature 검증에 gas 소비
ctx.GasMeter().ConsumeGas(params.SigVerifyCost, "signature verification")

// Tx 크기에 비례하는 gas 소비
txSizeGas := len(txBytes) * params.TxSizeCostPerByte
ctx.GasMeter().ConsumeGas(txSizeGas, "tx size")
```

Phase 4: Message Routing & Execution

```go
func (app *BaseApp) runMsgs(ctx sdk.Context, msgs []sdk.Msg, mode runTxMode) (*sdk.Result, error) {
    msgResults := make([]*sdk.Result, len(msgs))

    for i, msg := range msgs {
        // 메시지를 적절한 핸들러로 라우팅
        handler := app.msgServiceRouter.Handler(msg)
        if handler == nil {
            return nil, sdkerrors.Wrapf(sdkerrors.ErrUnknownRequest, "no handler for message %T", msg)
        }

        // 핸들러 실행
        msgResult, err := handler(ctx, msg)
        if err != nil {
            return nil, err
        }

        msgResults[i] = msgResult
    }

    // 모든 메시지 결과 병합
    return sdk.MergeResults(msgResults), nil
}
```

예시: Bank 모듈의 Send 메시지 처리

```go
// x/bank/keeper/msg_server.go
func (k msgServer) Send(goCtx context.Context, msg *types.MsgSend) (*types.MsgSendResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)

    // 1. 송신자와 수신자 주소 파싱
    from, err := sdk.AccAddressFromBech32(msg.FromAddress)
    to, err := sdk.AccAddressFromBech32(msg.ToAddress)

    // 2. 송금 실행
    err = k.SendCoins(ctx, from, to, msg.Amount)
    if err != nil {
        return nil, err
    }

    // 3. 이벤트 발생
    ctx.EventManager().EmitEvent(
        sdk.NewEvent(
            types.EventTypeTransfer,
            sdk.NewAttribute(types.AttributeKeyRecipient, msg.ToAddress),
            sdk.NewAttribute(sdk.AttributeKeyAmount, msg.Amount.String()),
        ),
    )

    return &types.MsgSendResponse{}, nil
}
```

Phase 5: State Changes

메시지 핸들러 내부에서 상태 변경:

```go
// KVStore에 쓰기
store := ctx.KVStore(k.storeKey)
key := types.BalanceKey(address, denom)
bz := k.cdc.MustMarshal(&balance)
store.Set(key, bz)
```

Phase 6: Event Emission

```
ctx.EventManager().EmitEvents(sdk.Events{
    sdk.NewEvent(
        sdk.EventTypeMessage,
        sdk.NewAttribute(sdk.AttributeKeyAction, types.TypeMsgSend),
        sdk.NewAttribute(sdk.AttributeKeySender, msg.FromAddress),
    ),
})
```

실전 코드 분석

BaseApp 초기화

```go
// app.go (실제 애플리케이션)
func NewMyApp(
    logger log.Logger,
    db dbm.DB,
    traceStore io.Writer,
    loadLatest bool,
    appOpts servertypes.AppOptions,
) *MyApp {
    // BaseApp 생성
    bApp := baseapp.NewBaseApp(appName, logger, db, txConfig.TxDecoder())

    // 앱 구조체 생성
    app := &MyApp{
        BaseApp: bApp,
        // ... 기타 필드
    }

    // ModuleManager 설정
    app.mm = module.NewManager(
        auth.NewAppModule(app.AccountKeeper),
        bank.NewAppModule(app.BankKeeper),
        staking.NewAppModule(app.StakingKeeper),
        // ... 기타 모듈
    )

    // ABCI 핸들러 설정
    app.SetInitChainer(app.InitChainer)
    app.SetBeginBlocker(app.BeginBlocker)
    app.SetEndBlocker(app.EndBlocker)

    // AnteHandler 설정
    anteHandler := auth.NewAnteHandler(
        app.AccountKeeper,
        app.BankKeeper,
        ante.DefaultSigVerificationGasConsumer,
    )
    app.SetAnteHandler(anteHandler)

    // Router 설정
    app.configureMsgServiceRouter()
    app.configureQueryRouter()

    // 상태 로드
    if loadLatest {
        err := app.LoadLatestVersion()
        if err != nil {
            panic(err)
        }
    }

    return app
}
```

InitChainer 구현

```go
func (app *MyApp) InitChainer(ctx sdk.Context, req abci.RequestInitChain) abci.ResponseInitChain {
    var genesisState GenesisState

    // Genesis state 언마샬링
    if err := json.Unmarshal(req.AppStateBytes, &genesisState); err != nil {
        panic(err)
    }

    // 각 모듈의 InitGenesis 호출
    app.mm.InitGenesis(ctx, app.appCodec, genesisState)

    // Validator 세트 반환
    return abci.ResponseInitChain{
        Validators: req.Validators,
    }
}
```

BeginBlocker & EndBlocker

```go
func (app *MyApp) BeginBlocker(ctx sdk.Context, req abci.RequestBeginBlock) abci.ResponseBeginBlock {
    return app.mm.BeginBlock(ctx, req)
}

func (app *MyApp) EndBlocker(ctx sdk.Context, req abci.RequestEndBlock) abci.ResponseEndBlock {
    return app.mm.EndBlock(ctx, req)
}
```

ModuleManager가 각 모듈의 BeginBlocker/EndBlocker를 순서대로 호출합니다.

다이어그램

ABCI 전체 플로우

```
┌──────────────────────────────────────────────────────────────┐
│                          CometBFT                            │
│  (Consensus, P2P, Mempool, Block Production)                 │
└──────────────────────────────────────────────────────────────┘
                  │                          │
                  │                          │
                  │ ABCI Interface           │
                  │                          │
      ┌───────────▼──────────────┐  ┌────────▼────────────────┐
      │   Consensus Connection   │  │   Mempool Connection    │
      │                          │  │                         │
      │  - InitChain             │  │  - CheckTx              │
      │  - BeginBlock            │  │                         │
      │  - DeliverTx             │  └─────────────────────────┘
      │  - EndBlock              │
      │  - Commit                │  ┌─────────────────────────┐
      │                          │  │   Info/Query Connection │
      │  ABCI++ additions:       │  │                         │
      │  - PrepareProposal       │  │  - Info                 │
      │  - ProcessProposal       │  │  - Query                │
      │  - ExtendVote            │  │                         │
      │  - VerifyVoteExtension   │  └─────────────────────────┘
      │  - FinalizeBlock         │
      └──────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────┐
│                          BaseApp                             │
│                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │ checkState   │  │ deliverState │  │  queryState  │       │
│  │ (mempool)    │  │ (execution)  │  │  (queries)   │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│                                                              │
│  ┌──────────────────────────────────────────────────┐        │
│  │                MsgServiceRouter                  │        │
│  │  /cosmos.bank.Msg/Send → bank.MsgServer.Send     │        │
│  └──────────────────────────────────────────────────┘        │
│                                                              │
│  ┌──────────────────────────────────────────────────┐        │
│  │                AnteHandler Chain                 │        │
│  │  Auth → Fee → Signature → Sequence               │        │
│  └──────────────────────────────────────────────────┘        │
└──────────────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────┐
│                      ModuleManager                           │
│                                                              │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐         │
│  │  auth   │  │  bank   │  │ staking │  │   gov   │         │
│  └─────────┘  └─────────┘  └─────────┘  └─────────┘         │
└──────────────────────────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────────────────────────┐
│                     CommitMultiStore                         │
│                                                              │
│  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐         │
│  │authStore│  │bankStore│  │stakStore│  │govStore │         │
│  │ (IAVL)  │  │ (IAVL)  │  │ (IAVL)  │  │ (IAVL)  │         │
│  └─────────┘  └─────────┘  └─────────┘  └─────────┘         │
└──────────────────────────────────────────────────────────────┘
```

트랜잭션 실행 플로우 (DeliverTx)

```
                    DeliverTx(txBytes)
                           │
                           ▼
                 ┌─────────────────────┐
                 │   1. Tx Decoding    │
                 │   txBytes → Tx      │
                 └─────────────────────┘
                           │
                           ▼
                 ┌─────────────────────┐
                 │  2. Context Setup   │
                 │  - deliverState.ctx │
                 │  - GasMeter         │
                 └─────────────────────┘
                           │
                           ▼
        ┌──────────────────────────────────┐
        │      3. AnteHandler Chain        │
        │                                  │
        │  SetUpContext                    │
        │      ↓                           │
        │  ValidateBasic                   │
        │      ↓                           │
        │  DeductFee (gas fee 차감)        │
        │      ↓                           │
        │  SetPubKey                       │
        │      ↓                           │
        │  SigVerification (서명 검증)     │
        │      ↓                           │
        │  IncrementSequence (nonce++)     │
        │                                  │
        └──────────────────────────────────┘
                     │
                     ▼
           ┌────────────────────┐
           │  4. Message Routing │
           │  MsgServiceRouter   │
           └────────────────────┘
                     │
      ┌──────────────┼──────────────┐
      │              │              │
      ▼              ▼              ▼
  ┌─────────┐    ┌─────────┐    ┌─────────┐
  │  Msg 1  │    │  Msg 2  │    │  Msg 3  │
  │ Handler │    │ Handler │    │ Handler │
  └─────────┘    └─────────┘    └─────────┘
      │              │              │
      └──────────────┼──────────────┘
                     │
                     ▼
        ┌────────────────────┐
        │   5. State Changes  │
        │   - KVStore writes  │
        │   - Balance updates │
        │   - Module state    │
        └────────────────────┘
                     │
                     ▼
        ┌────────────────────┐
        │   6. Event Emission │
        │   - EventManager    │
        │   - Events for each │
        │     message         │
        └────────────────────┘
                     │
                     ▼
           ┌────────────────────┐
           │  7. Response Return │
           │  - GasUsed          │
           │  - Events           │
           │  - Data             │
           └────────────────────┘
```

Block Lifecycle (ABCI++)

```
Block Height N-1 Committed
         │
         ▼
┌──────────────────────────────────────┐
│        Start Block Height N          │
└──────────────────────────────────────┘
            │
            ▼
      ┌──────────────┐
      │ [Proposer]   │
      │              │
      │ Prepare      │──────┐
      │ Proposal     │      │ Select & Order Txs
      └──────────────┘      │ from Mempool
            │               │
            │◄──────────────┘
            │ Ordered TxSet
            ▼
      ┌──────────────┐
      │ [Validators] │
      │              │
      │  Process     │──────┐
      │  Proposal    │      │ Validate TxSet
      └──────────────┘      │ Accept or Reject
            │               │
            │◄──────────────┘
            │ Accept/Reject
            ▼
      ┌──────────────┐
      │ [Consensus]  │
      │              │
      │  Prevote     │──────┐
      │              │      │ 2/3+ Prevotes
      └──────────────┘      │
            │               │
            │◄──────────────┘
            ▼
      ┌──────────────┐
      │ [Validators] │
      │              │
      │ Extend       │──────┐
      │ Vote         │      │ Add oracle data,
      └──────────────┘      │ price feeds, etc.
            │               │
            │◄──────────────┘
            │ Vote + Extension
            ▼
      ┌──────────────┐
      │ [Validators] │
      │              │
      │  Verify Vote │──────┐
      │  Extension   │      │ Validate extensions
      └──────────────┘      │
            │               │
            │◄──────────────┘
            ▼
      ┌──────────────┐
      │ [Consensus]  │
      │              │
      │  Precommit   │──────┐
      │              │      │ 2/3+ Precommits
      └──────────────┘      │
            │               │
            │◄──────────────┘
            ▼
┌──────────────────────────┐
│      Finalize Block      │
│                          │
│  ┌────────────────────┐  │
│  │   BeginBlock Logic │  │
│  │  - Mint rewards    │  │
│  │  - Slash misbehavior│  │
│  └────────────────────┘  │
│          │               │
│          ▼               │
│  ┌────────────────────┐  │
```

```
│  │   Execute Each Tx      │  │
│  │  - Decode              │  │
│  │  - AnteHandler         │  │
│  │  - Msg Routing         │  │
│  │  - State Changes       │  │
│  └────────────────────────┘  │
│           │                  │
│           ▼                  │
│  ┌────────────────────────┐  │
│  │   EndBlock Logic       │  │
│  │  - Validator updates   │  │
│  │  - Module EndBlockers  │  │
│  └────────────────────────┘  │
│           │                  │
└───────────┴──────────────────┘
            │
            ▼
      ┌──────────────┐
      │    Commit    │
      │              │
      │ - Write to   │
      │   disk       │
      │ - Calculate  │
      │   AppHash    │
      │ - Prune old  │
      │   versions   │
      └──────────────┘
            │
            ▼
  Block Height N Committed
```

Best Practices

1. ABCI 메소드 구현 시

✅ DO

- **멱등성 보장**: 같은 입력에 대해 항상 같은 결과
- **결정론적 실행**: 모든 validator가 같은 결과를 얻어야 함
- **효율적인 로직**: 블록 생성 시간에 영향
- **명확한 에러 처리**: 적절한 에러 코드 반환

❌ DON'T

- **비결정론적 요소 사용 금지**: `time.Now()`, `rand.Random()`, map iteration
- **외부 API 호출 금지**: HTTP 요청, 데이터베이스 쿼리 (상태 외부)
- **무거운 연산**: BeginBlock/EndBlock에서 과도한 계산
- **상태 누수**: checkState와 deliverState 혼동

2. 비결정론적 코드 피하기

```go
// ❌ 잘못된 예
func (k Keeper) ProcessBlock(ctx sdk.Context) {
    // 현재 시간 사용 - 각 validator마다 다를 수 있음
    now := time.Now()

    // Map iteration - 순서가 보장되지 않음
    for k, v := range myMap {
        // ...
    }

    // 외부 API 호출
    resp, _ := http.Get("https://api.example.com/data")
}

// ✅ 올바른 예
func (k Keeper) ProcessBlock(ctx sdk.Context) {
    // 블록 시간 사용 (합의된 값)
    blockTime := ctx.BlockTime()

    // Slice로 변환 후 정렬
    keys := make([]string, 0, len(myMap))
    for k := range myMap {
        keys = append(keys, k)
    }
    sort.Strings(keys)
    for _, k := range keys {
        v := myMap[k]
        // ...
    }

    // 상태에서 읽기 (오라클 데이터는 이전에 저장됨)
    data := k.GetStoredData(ctx)
}
```

3. Gas 관리

```go
// Gas 소비 명시
ctx.GasMeter().ConsumeGas(gasAmount, "operation description")

// Gas 한도 체크
if ctx.GasMeter().IsOutOfGas() {
    panic(sdk.ErrorOutOfGas{Descriptor: "my operation"})
}

// Gas 리펀드 (storage 삭제 시)
ctx.GasMeter().RefundGas(refundAmount, "storage deletion")
```

4. Context 관리

```go
// Context는 immutable - 수정 시 새 context 반환
newCtx := ctx.WithBlockHeight(newHeight)

// Context chaining
ctx = ctx.
    WithBlockHeight(100).
    WithBlockTime(time.Now()).
    WithChainID("my-chain")

// Cache context 사용 (롤백 가능)
cacheCtx, writeFn := ctx.CacheContext()
// ... 연산 수행
if success {
    writeFn()  // cache를 실제 context에 적용
}
// 실패 시 writeFn() 호출하지 않으면 자동 롤백
```

5. State 접근 패턴

```go
// Keeper를 통한 state 접근
type Keeper struct {
    storeKey sdk.StoreKey
    cdc      codec.BinaryCodec
}

func (k Keeper) GetBalance(ctx sdk.Context, addr sdk.AccAddress, denom string) sdk.Coin {
    store := ctx.KVStore(k.storeKey)
    key := types.BalanceKey(addr, denom)

    bz := store.Get(key)
    if bz == nil {
        return sdk.NewCoin(denom, sdk.ZeroInt())
    }

    var balance sdk.Coin
    k.cdc.MustUnmarshal(bz, &balance)
    return balance
}

func (k Keeper) SetBalance(ctx sdk.Context, addr sdk.AccAddress, balance sdk.Coin) {
    store := ctx.KVStore(k.storeKey)
    key := types.BalanceKey(addr, balance.Denom)

    bz := k.cdc.MustMarshal(&balance)
    store.Set(key, bz)
}
```

참고 자료

공식 문서

- CometBFT ABCI Specification
- CometBFT ABCI++ Spec
- Cosmos SDK BaseApp Documentation

소스 코드

- BaseApp 구현
- ABCI 인터페이스 정의
- ABCI 타입 정의 (Protobuf)

추가 학습 자료

- ABCI++ 소개 블로그 포스트
- Cosmos SDK ADR-028: Public Key Addresses

다음 단계

ABCI 아키텍처를 이해했다면, 다음 주제로 넘어갈 수 있습니다:

1. **Module System** → module-system.md
    - Module Manager의 역할
    - AppModule 인터페이스
    - BeginBlocker/EndBlocker 구현
2. **State Management** → state-management.md
    - KVStore 계층 구조
    - IAVL tree 원리
    - MultiStore 아키텍처
3. **Transaction Lifecycle** → transaction-lifecycle.md
    - 트랜잭션 생성부터 실행까지 전체 추적
    - Gas 계산 메커니즘
    - 실습: Transaction lifecycle tracer 개발

작성자: B-Harvest Core Developer Candidate **버전**: 1.0 **최종 수정일**: 2025-01-XX